

Pipeline CI/CD com Docker

Projeto Cidades ESG Inteligentes
API Eficiência Energética (ESG)

Integrantes	RM
Arthur Wylliam Santos Brandão	RM562519
Carlos Antonio Campos Machado	RM562904
Jennyfer Barbosa de Souza	RM565354
Nargila Soares Mota	RM562956
Pedro Felipe Colaço De Biazzi	RM553309

Repositório: <https://github.com/JennyBarbosa22k/cidades-esg-inteligentes>

Staging: <https://esg-staging.onrender.com/swagger>

Produção: <https://esg-production.onrender.com/swagger>

Stack: C# / .NET 8, PostgreSQL, Docker, Docker Compose, GitHub Actions, Render.

1. Descrição do pipeline

Ferramenta: GitHub Actions (arquivo `.github/workflows/ci-cd.yml`). O pipeline automatiza o ciclo de vida da API em .NET 8: compila o projeto, executa os testes, valida a imagem Docker e publica a aplicação nos ambientes de **staging** e **produção**, hospedados no Render.

1. Build e Testes
restore, build, test

2. Imagem Docker
docker build

3. Deploy Staging
hook + smoke test

4. Deploy Produção
aprovação manual

Cada job só inicia se o anterior for concluído com sucesso.

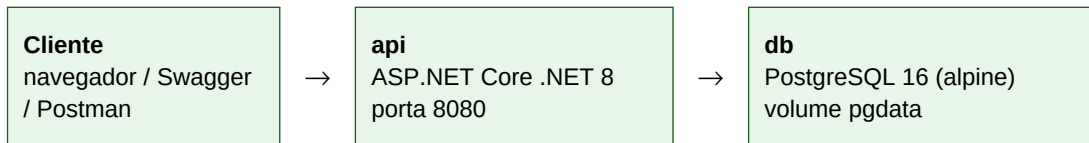
Job	Lógica	Gatilho
Build e Testes	<i>dotnet restore</i> , <i>dotnet build -c Release</i> e <i>dotnet test</i> : executa os 10 testes de integração xUnit (WebApplicationFactory com banco InMemory).	push e pull request na main
Build da imagem Docker	<i>docker build</i> a partir do Dockerfile, validando a containerização antes de qualquer deploy.	após build e testes
Deploy Staging	Aciona o <i>Deploy Hook</i> do Render (secret RENDER_DEPLOY_HOOK do environment staging) e faz teste de fumaça em <i>/health</i> com novas tentativas.	push na main
Deploy Produção	Depende do staging. O environment <i>production</i> tem <i>Required reviewers</i> : o deploy só ocorre após aprovação manual. Depois, teste de fumaça em <i>/health</i> .	push na main, após staging

Gatilhos: push e pull request na branch main, além de execução manual (`workflow_dispatch`). Em pull requests rodam apenas os jobs de build, testes e imagem; os deploys são ignorados.

Observação: o job de imagem valida o build do Dockerfile. Nos ambientes, o Render constrói a imagem a partir do mesmo Dockerfile ao receber o Deploy Hook.

2. Docker

2.1 Arquitetura



Os dois serviços ficam na rede **energia-net** (bridge); a API acessa o banco pelo nome **db**.

Componente	Detalhe
api	Construída do Dockerfile. Variáveis: ConnectionStrings__DefaultConnection, Jwt__Key, ASPNETCORE_ENVIRONMENT. Porta publicada em APP_PORT (padrão 8080).
db	postgres:16-alpine com healthcheck (<i>pg_isready</i>); dados persistidos no volume pgdata .
Rede	energia-net (bridge).
Variáveis de ambiente	Definidas no arquivo .env (modelo em .env.example): POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, JWT_KEY, APP_PORT.
Ordem de subida	A api depende do db com <i>condition: service_healthy</i> .
Staging / produção	O Render constrói a imagem do mesmo Dockerfile e injeta as variáveis. Há um único PostgreSQL gratuito, compartilhado entre os ambientes e isolado em schemas diferentes (staging e producao).

2.2 Imagem criada (Dockerfile multi-stage)

Estágio	Base	Função
build	mcr.microsoft.com/dotnet/sdk:8.0	Restaura dependências (camada em cache) e publica em Release.
final	mcr.microsoft.com/dotnet/aspnet:8.0	Imagem enxuta só com o runtime e os binários publicados; executa com usuário não-root (USER app) na porta 8080.

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY src/EnergiaSustentavel.API/EnergiaSustentavel.API.csproj src/EnergiaSustentavel.API/
RUN dotnet restore src/EnergiaSustentavel.API/EnergiaSustentavel.API.csproj
COPY src/ src/
RUN dotnet publish src/EnergiaSustentavel.API/EnergiaSustentavel.API.csproj \
    -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
COPY --from=build /app/publish .
USER app
ENTRYPOINT ["dotnet", "EnergiaSustentavel.API.dll"]
```

2.3 Comandos utilizados

```
cp .env.example .env                # ajustar senha do banco e chave JWT
docker compose up -d --build         # constrói a imagem e sobe API + PostgreSQL
docker compose ps                    # status dos serviços
docker compose logs -f api           # logs da API
curl http://localhost:8080/health    # health check -> Healthy
                                     # Swagger: http://localhost:8080/swagger
docker compose down                  # encerra (-v apaga também o volume)

docker build -t energia-sustentavel:local . # build isolado da imagem (usado no pipeline)
dotnet test                           # executa os testes xUnit
```

Autenticação: os endpoints de escrita exigem JWT. Faça POST /api/auth/login, copie o token e use o botão Authorize do Swagger.

3. Prints do pipeline rodando

As imagens abaixo são da execução #21 do pipeline, concluída com sucesso em 5m30s.

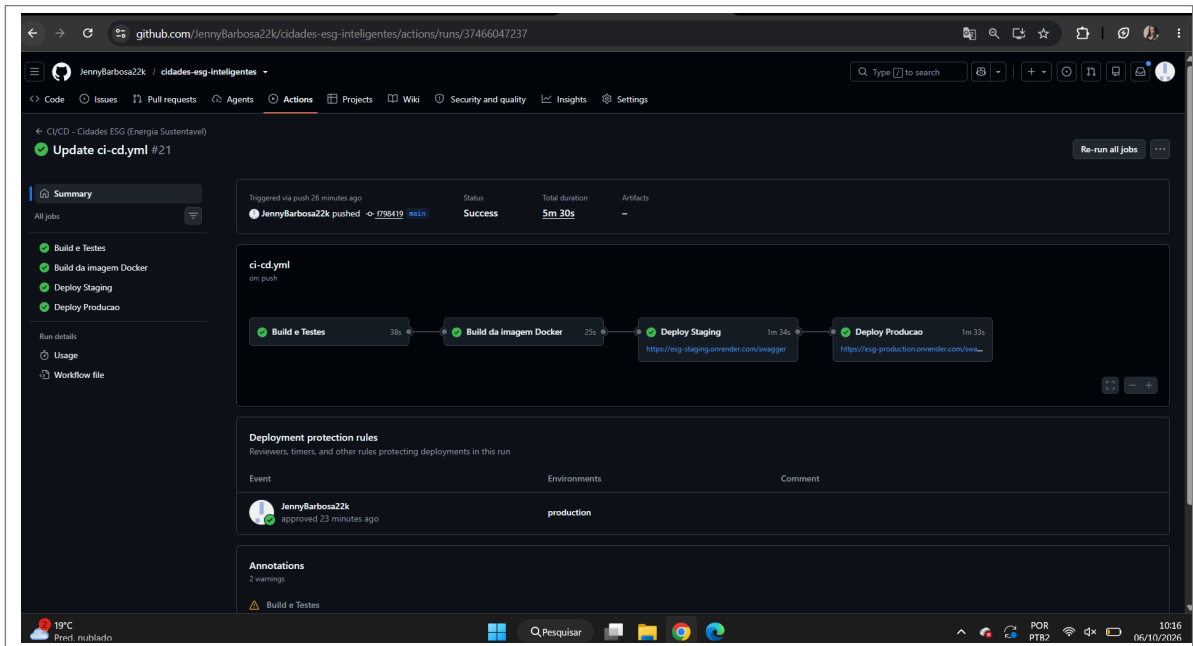


Figura 1 - Pipeline completa: os quatro jobs concluídos e a aprovação manual do ambiente production (Deployment protection rules).

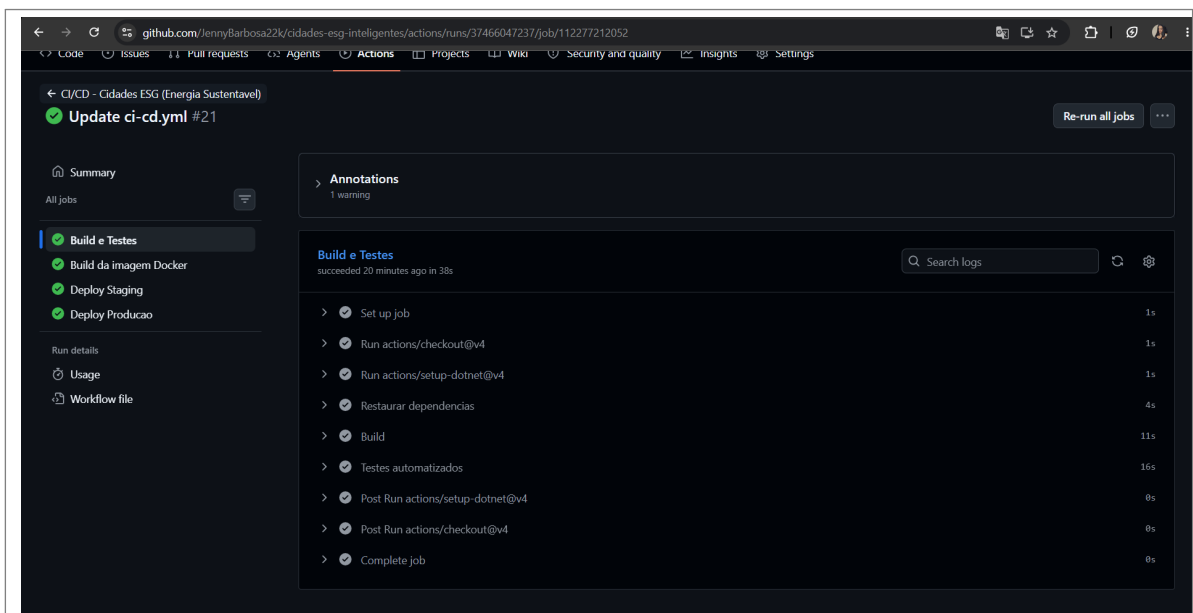


Figura 2 - Job Build e Testes: restauração, build e testes automatizados concluídos.

```

217 info: Microsoft.AspNetCore.Mvc.Infrastructure.ObjectResultExecutor[1]
218   Executing OkObjectResult, writing value of type 'EnergiaSustentavel.API.ViewModels.ResultadoPaginado'1[[EnergiaSustentavel.API.ViewModels.EquipamentoViewModel, EnergiaSustentavel.API, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null]]'.
219 info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[105]
220   Executed action EnergiaSustentavel.API.Controllers.EquipamentosController.Listar (EnergiaSustentavel.API) in 1.4485ms
221 info: Microsoft.AspNetCore.Routing.EndpointMiddleware[1]
222   Executed endpoint 'EnergiaSustentavel.API.Controllers.EquipamentosController.Listar (EnergiaSustentavel.API)'
223 info: Microsoft.AspNetCore.Hosting.Diagnostics[2]
224   Request finished HTTP/1.1 GET http://localhost/api/equipamentos?pagina=1&tamanhoPagina=5 - 200 - application/json;charset=utf-8 3.2246ms
225 info: Microsoft.AspNetCore.Mvc.Infrastructure.ObjectResultExecutor[1]
226   Executing OkObjectResult, writing value of type 'EnergiaSustentavel.API.ViewModels.TokenViewModel'.
227 info: Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker[105]
228   Executed action EnergiaSustentavel.API.Controllers.AuthController.Login (EnergiaSustentavel.API) in 417.6212ms
229 info: Microsoft.AspNetCore.Routing.EndpointMiddleware[1]
230   Executed endpoint 'EnergiaSustentavel.API.Controllers.AuthController.Login (EnergiaSustentavel.API)'
231 info: Microsoft.AspNetCore.Hosting.Diagnostics[2]
232   Request finished HTTP/1.1 POST http://localhost/api/auth/login - 200 - application/json;charset=utf-8 419.3044ms
233 [xUnit.net 00:00:06.71] Finished: EnergiaSustentavel.Tests
234 Passed EnergiaSustentavel.Tests.AlertasControllerTests.Get_ReturnsHttpStatus200 [43 ms]
235 Passed EnergiaSustentavel.Tests.ConsumoControllerTests.GetResumo_ReturnsHttpStatus200 [44 ms]
236 Passed EnergiaSustentavel.Tests.LeiturasControllerTests.Get_ReturnsHttpStatus200 [50 ms]
237 Passed EnergiaSustentavel.Tests.LoginControllerTests.Login_ComCredenciaisInvalidas_ReturnsUnauthorized [5 s]
238 Passed EnergiaSustentavel.Tests.EquipamentosControllerTests.Get_ReturnsHttpStatus200 [467 ms]
239 Passed EnergiaSustentavel.Tests.EquipamentosControllerTests.Get_ComPaginacao_ReturnsHttpStatus200 [5 ms]
240 Passed EnergiaSustentavel.Tests.AuthControllerTests.Login_ComCredenciaisValidas_ReturnsHttpStatus200 [422 ms]
241
242 Test Run Successful.
243 Total tests: 10
244 Passed: 10
245 Total time: 13.4518 Seconds
246
247 Done Building Project "/home/runner/work/cidades-esg-inteligentes/cidades-esg-inteligentes/EnergiaSustentavel.sln" (VSTest target(s)).
248 Build succeeded.
249 0 Warning(s)
250 0 Error(s)
251
252 Time Elapsed: 00:00:15.61

```

Figura 3 - Saída do passo Testes automatizados: 10 testes executados e 10 aprovados (Passed: 10).

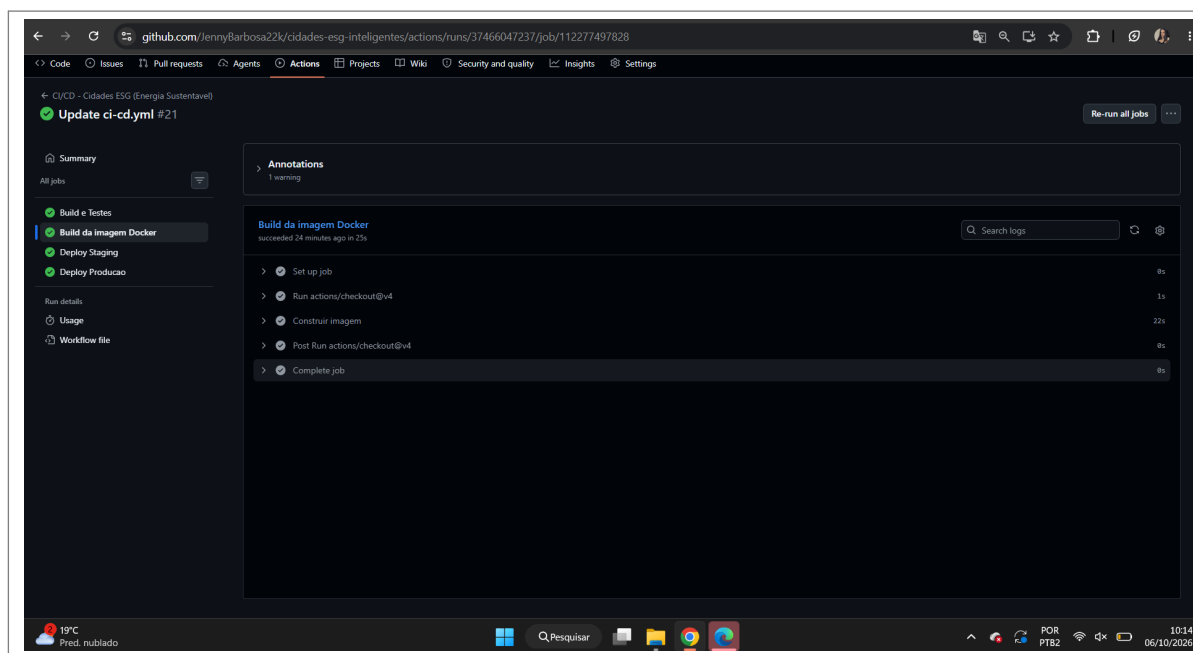


Figura 4 - Job Build da imagem Docker concluído com sucesso (passo Construir imagem).

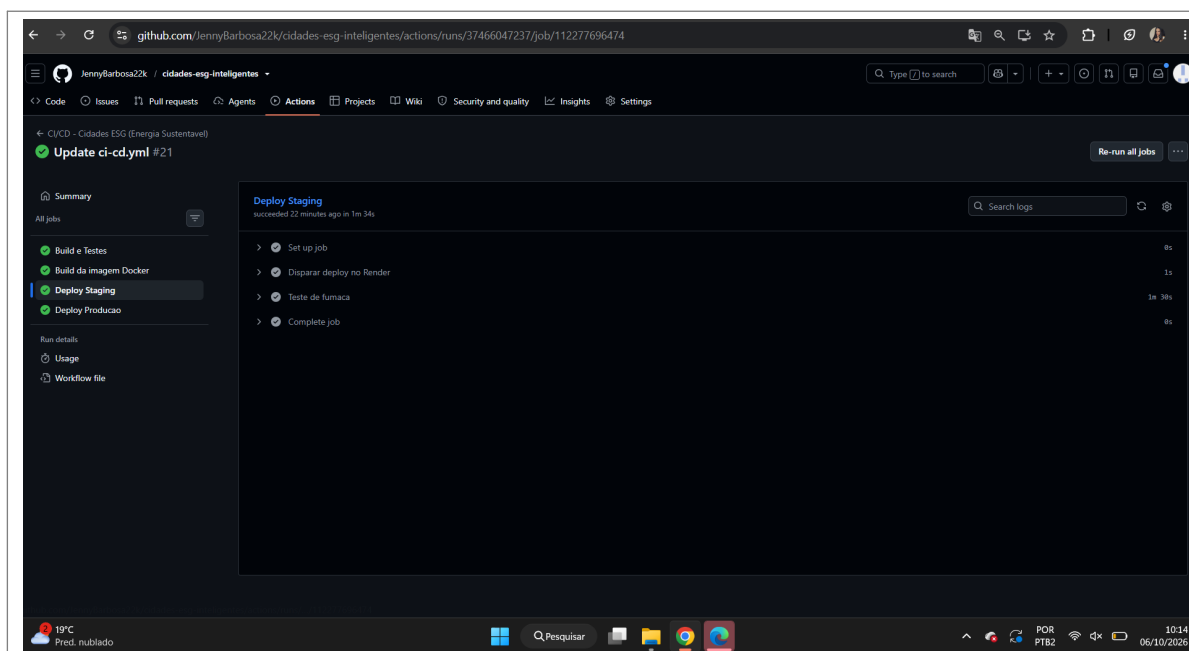


Figura 5 - Job Deploy Staging: disparo do deploy no Render e teste de fumaça concluídos.

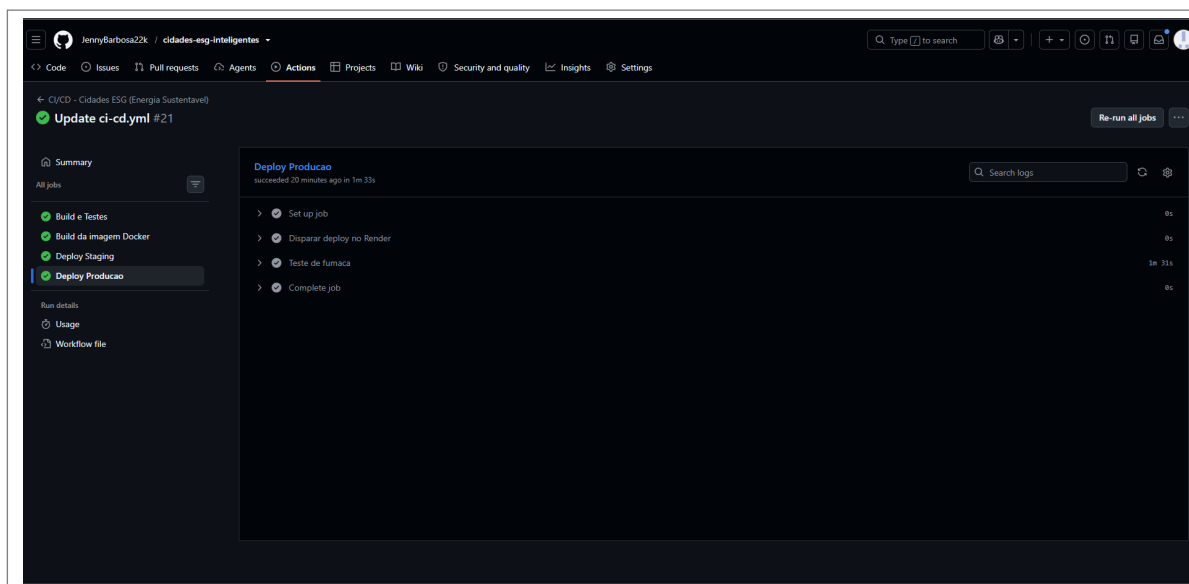


Figura 6 - Job Deploy Produção: executado após a aprovação manual, com teste de fumaça concluído.

4. Prints dos ambientes staging e produção

Staging - <https://esg-staging.onrender.com/swagger>

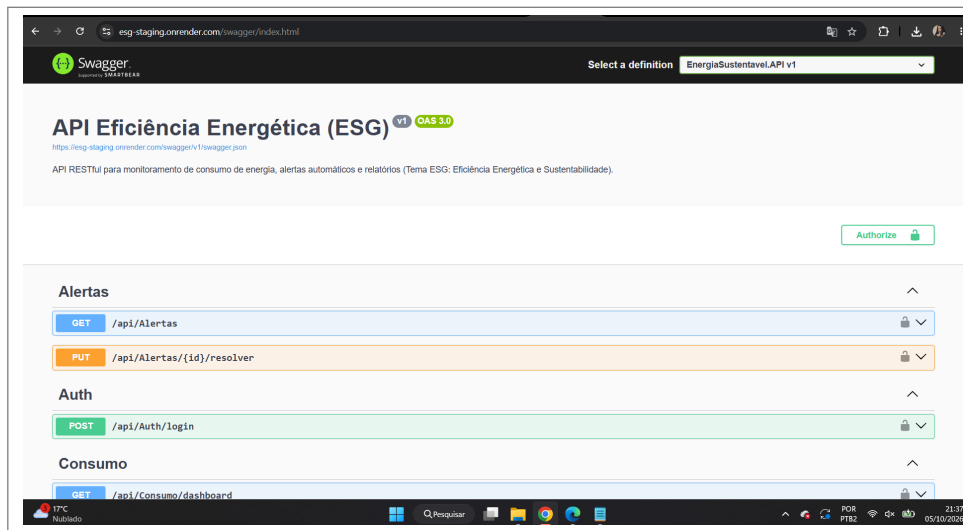


Figura 7 - Swagger da API em staging.



Figura 8 - Endpoint /health de staging respondendo Healthy (recorte da captura original).

Produção - <https://esg-production.onrender.com/swagger>

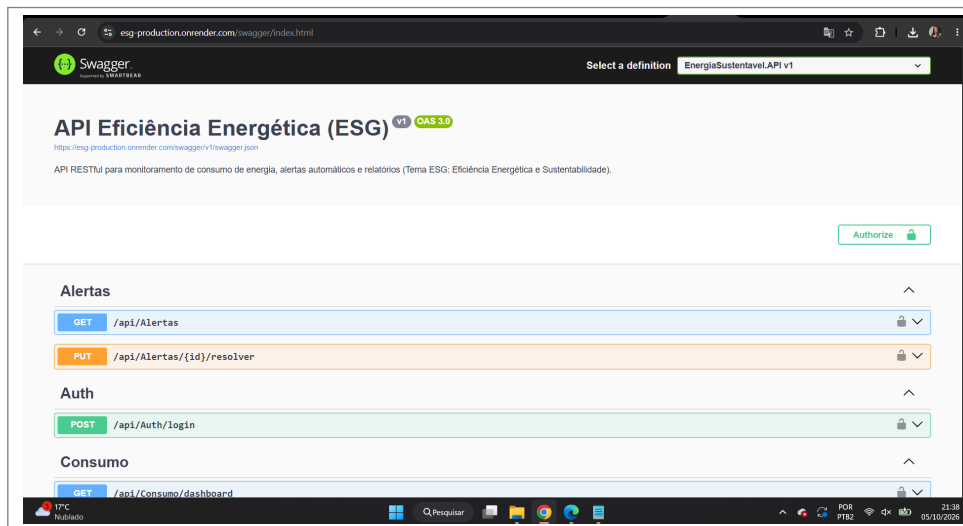


Figura 9 - Swagger da API em produção.

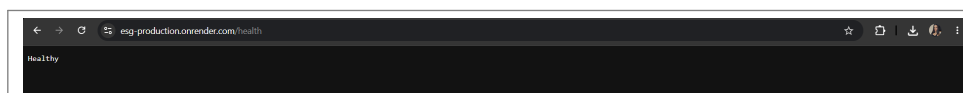


Figura 10 - Endpoint /health de produção respondendo Healthy (recorte da captura original).

5. Desafios encontrados e como foram resolvidos

Desafio	Solução
Testes de login falhando no pipeline (401 Unauthorized). O teste Login_ComCredenciaisValidas falhava, enquanto os outros nove passavam. Na CustomWebApplicationFactory, o nome do banco em memória era gerado dentro da função de configuração do DbContext, criando um banco diferente a cada requisição: o seed gravava o usuário em um banco e o login consultava outro, vazio.	O nome do banco passou a ser gerado uma única vez, antes do ConfigureServices, para que todas as requisições usem o mesmo banco.
Workflow não reconhecido pelo GitHub. Após uma alteração, o ci-cd.yml ficou apenas com o trecho de um job, sem os campos name, on e jobs. O GitHub não listava o workflow e a execução falhava em 0s.	A estrutura completa do arquivo, com gatilhos e jobs, foi restaurada a partir do histórico de commits.
Perda de parte da pipeline. Era preciso recuperar a versão anterior com os jobs de imagem Docker e de deploy.	O histórico do arquivo no GitHub permitiu recuperar a versão anterior, evitando reescrever o pipeline do zero.
Um único banco PostgreSQL no plano gratuito do Render para dois ambientes.	Staging e produção compartilham a instância, isolados em schemas diferentes (variável Database__Schema).

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	[X]
Dockerfile funcional	[X]
docker-compose.yml ou arquivos Kubernetes	[X]
Pipeline com etapas de build, teste e deploy	[X]
README.md com instruções e prints	[X]
Documentação técnica com evidências (PDF ou PPT)	[X]
Deploy realizado nos ambientes staging e produção	[X]

Repositório: <https://github.com/JennyBarbosa22k/cidades-esg-inteligentes>